

MITS 6700G - Assignment 1

Avishkar Sharna

Ontario Tech University

Complex Networks - MITS 6700G

7th Jun 2026

Contents

Question Number 1	1
Social Network Analysis for 4 networks:.....	1
Metric Computation for 1 minute Conversation:	2
Metric Computation for 7 Min Conversation:	5
Metric Computation for Contact Information:	8
Metric Computation for Overall Eye Contact:.....	11
Cross-Network Insights emerge when examining the networks:.....	14
What Would Be Lost If the Networks Were Examined Individually.....	14
Question Number 2:.....	16
Libraries used:.....	16
High level flow-chart of code:	17
Full Source Code for crawler :.....	18
Metrics to Report:	30
Degree Distribution:	30
Connected Component Report:.....	31
Closeness Centrality Distribution:.....	32
Betweenness Centrality Distribution:	33
Page Rank Distribution:	34
Top 10 Most and Least central Webpages:.....	35
Interpretation:	37
Visualization of web Crawled using Gephi:	39
Appendix:	40

Question Number 1

Social Network Analysis for 4 networks:

1-Min Conversation:

Justification for Directed and Unweighted:

Property	Value
Directed / Undirected	Directed
Weighted/Unweighted	Unweighted
Justification	Directed because A may talk to B without B talking to A. unweighted because 1.0 = one way and having two 1.0 in a single edge means reciprocal.

Network for 1 Minute Conversation based on Degree

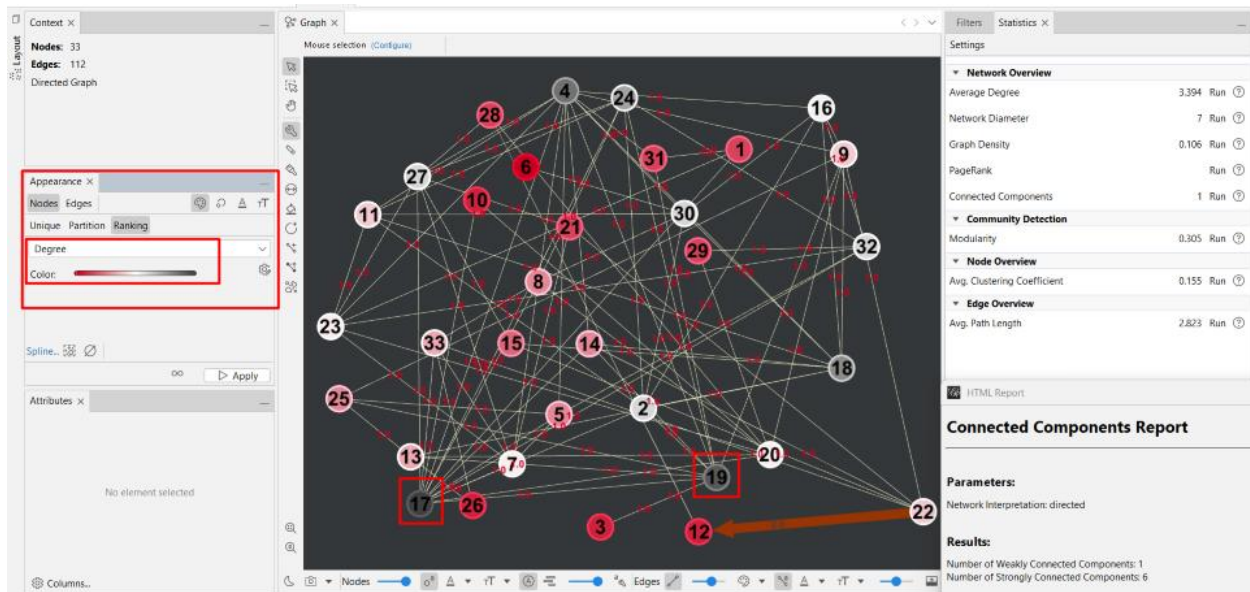


Figure 1: Network for 1 Minute Conversation based on Degree

Network for 1 Minute Conversation based on Betweenness Centrality

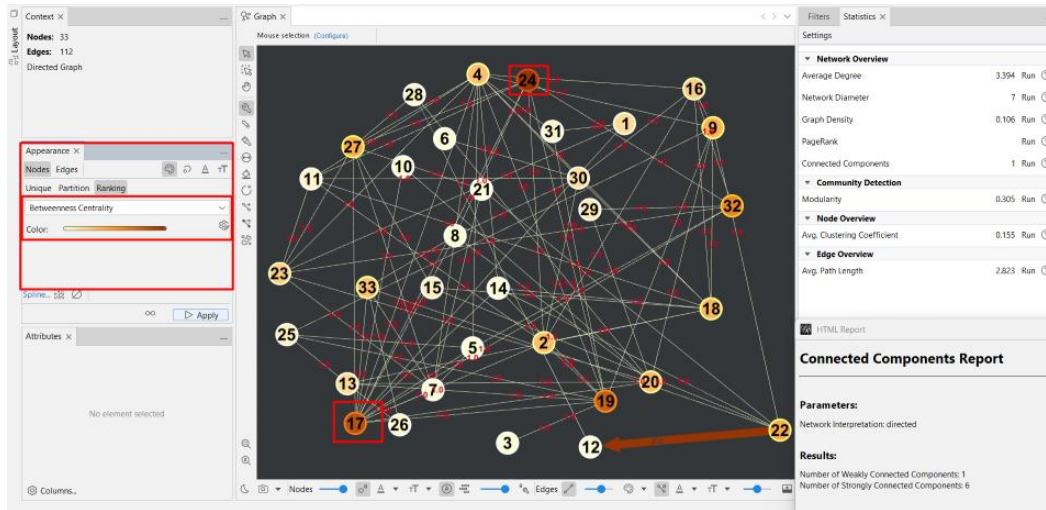


Figure 2 : Network for 1 Minute Conversation based on Betweenness Centrality

Metric Computation for 1 minute Conversation:

Metric	Value
Number of Nodes	33
Number of Edges	112
Edge Density	0.106
Degree distribution	3.394
Average clustering coefficient	0.155
Number of nodes in SCC and WCC	Weakly Connected Component = 1 Strongly Connected Components = 8
Average path length (SCC)	2.823
Network Diameter	7
Top 2 By Degree	19 and 17
Top 2 By Betweenness Centrality	17 and 24

Table 1: Overall Graph Metrics (Class-Wide) for 1 Minute Conversations

Insights for Overall Metrics (1 Min Conversation):

The observed graph of the 1-minute conversation is sparse, with just 0.106 density. It implies that a few students were talking to only a very few members of the class. The WCC at 1 and SCC at 8 signify one-way conversation for the most part, rather than mutual. This results in an average path length of 2.82, where any two students get connected in 3 interactions on average. The top 2 degree are Node 19, Salman Kuthpudeen, and Node 17, Binayak Kshetri. The top 2 betweenness are Node 17, Binayak Kshetri, and Node 24, Samir Rahman

My personal Network for 1 Minute Conversation

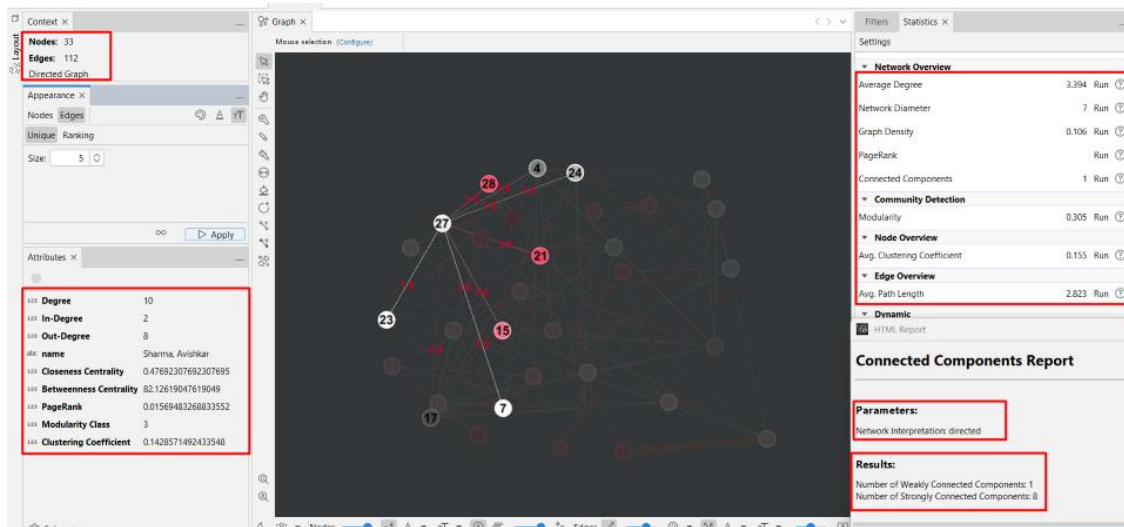


Figure 3 : My personal Network for 1 Minute Conversation

Metric computation for personal Network for 1 Minute Conversation

Metric	My Value
In-Degree	2
Out-Degree	8
Total Degree	10
Closeness Centrality	0.476
Betweenness Centrality	82.123
Page Rank	0.051
Community (Modularity Class)	3

Table 2: My Personal Metrics for 1 Minute Conversations

Insights for Personal Metrics (1 Min Conversation):

For my personal metrics, as seen in above graph, I initiated conversation with as many as 8 students, but few initiated with me, so I have in-degree of 2. However, my betweenness is 82 means I occasionally connect different groups.

7 min Conversation:

Property	Value
Directed / Undirected	Directed
Weighted/Unweighted	Unweighted
Justification	Directed because A may talk to B without B talking to A. unweighted because 1.0 = one way and having two 1.0 in a single edge means reciprocal.

Network for 7 Minute Conversation Based on Closeness Centrality

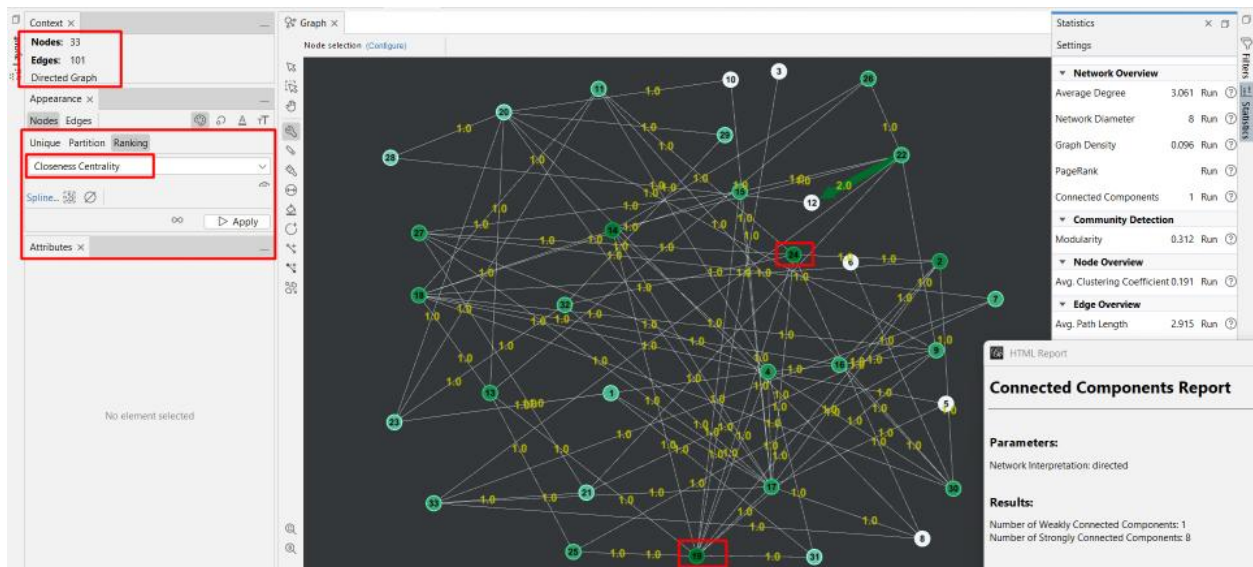


Figure 4 : Network for 7 Minute Conversation Based on Closeness Centrality

Network for 7 Minute Conversation Based on Clustering Coefficient



Figure 5 : Network for 7 Minute Conversation Based on Clustering Coefficient

Metric Computation for 7 Min Conversation:

Metric	Value
Number of Nodes	33
Number of Edges	101
Edge Density	0.096
Degree distribution	3.061
Average clustering coefficient	0.191
Number of nodes in SCC and WCC	Weakly Connected Component = 1 Strongly Connected Components = 8
Average path length (SCC)	2.915
Network Diameter	8
Top 2 By Closeness Centrality	19 and 24
Top 2 By Clustering Coefficient	25 and 5

Table 3: Overall (Class-Wide) Metrics for 7 Minute Conversations

Insights for Overall Metrics (7 Min Conversation):

The network graph for 7 Minute Conversations is even sparser, only with a density of 0.096 compared to the 1-minute network. This comes with Weak Connected Component: 1 and Strong Connected Component: 8. A higher diameter of 8 would mean more steps across the class to connect. The top 2 by closeness centrality are Node 19 - Salmaan Kuthpudeen and Node 24 - Samir Rahman; in other words, they can easily reach others. The top 2 by clustering coefficient are Node 25 - Vishnu Prasath Sathya Narayanan and Node 5 - Rithika Biju.

My Personal Network for 7 Minute Conversation



Figure 6 : My Personal Network for 7 Minute Conversation

Metric Computation for Personal connection for 7 Minute Conversations:

Metric	My Value
In-Degree	2
Out-Degree	8
Total Degree	10
Closeness Centrality	0.476
Betweenness Centrality	82.123
Page Rank	0.051
Community (Modularity Class)	3

Table 4 : My Personal Metrics for 7 Minute Conversations

Insights for Personal Metrics (7 Min Conversation):

I actively initiate conversations with many classmates, so I have out degree of 8, but few initiate back with me so I have in-degree of 2. My betweenness of 82.12 and closeness of 0.476 show I am well-positioned to reach others quickly.

Contact Information:

Property	Value
Directed / Undirected	Directed
Weighted/Unweighted	Unweighted
Justification	Contact sharing is directed because it can be one-way. Its unweighted because we only track whether contact information exists in each direction, not the quantity of method shared.

Network for Contact Information based on Betweenness Centrality



Figure 7 : Network for Contact Information based on Betweenness Centrality

Network for Contact Information based on Closeness Centrality:

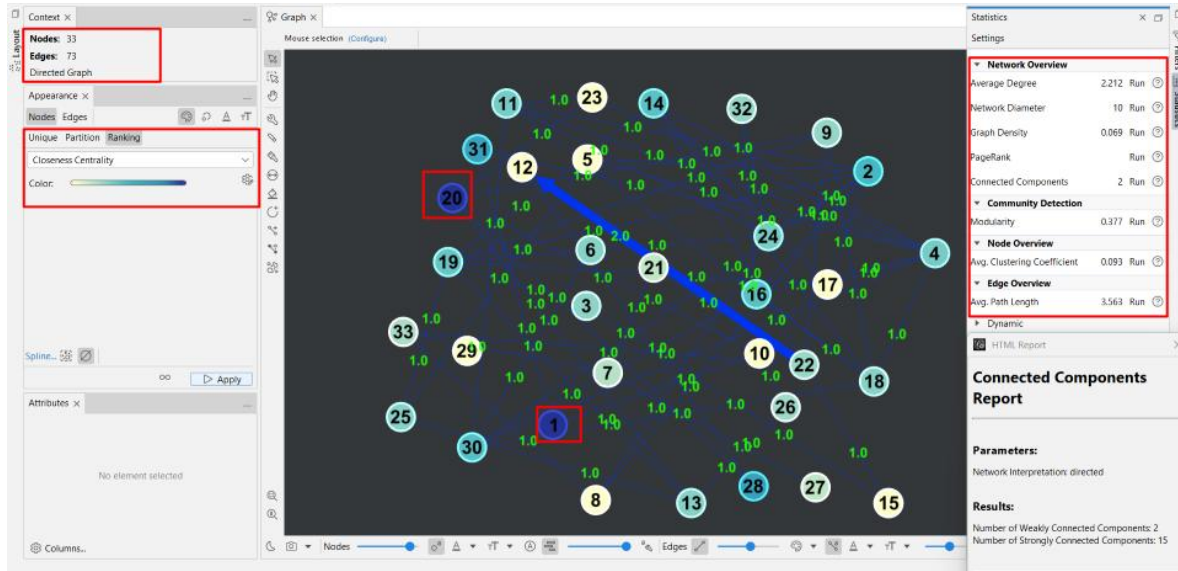


Figure 8 : Network for Contact Information based on Closeness Centrality

Metric Computation for Contact Information:

Metric	Value
Number of Nodes	33
Number of Edges	73
Edge Density	0.069
Degree distribution	2.212
Average clustering coefficient	0.093
Number of nodes in SCC and WCC	Weakly Connected Components = 2 Strongly Connected Components = 15
Average path length (SCC)	3.563
Network Diameter	10
Top 2 By Betweenness Centrality	19 and 30
Top 2 By Closeness Centrality	20 and 1

Table 5 : Overall (Class-Wide) Metrics for Contacts Information'

Insights for Overall Metrics (Contacts Information):

Contact information network density is 0.069, so it has a very sparse network where very few students share their contact information. It has 2 Weakly Connected Components (WCC), and therefore at least one student or group is completely isolated. The high diameter of 10, and a longer path length of 3.56 reveal fragmentation in the contact information network. The top 2 by betweenness are node 19 (Salmaan Kuthpudeen) and node 30 (Asma Syed), while the top 2 by closeness are node 20 (Mohammadsaad Langawala) and node 1 (Tashkin Ahmed).

My Personal Network for Contact Information:

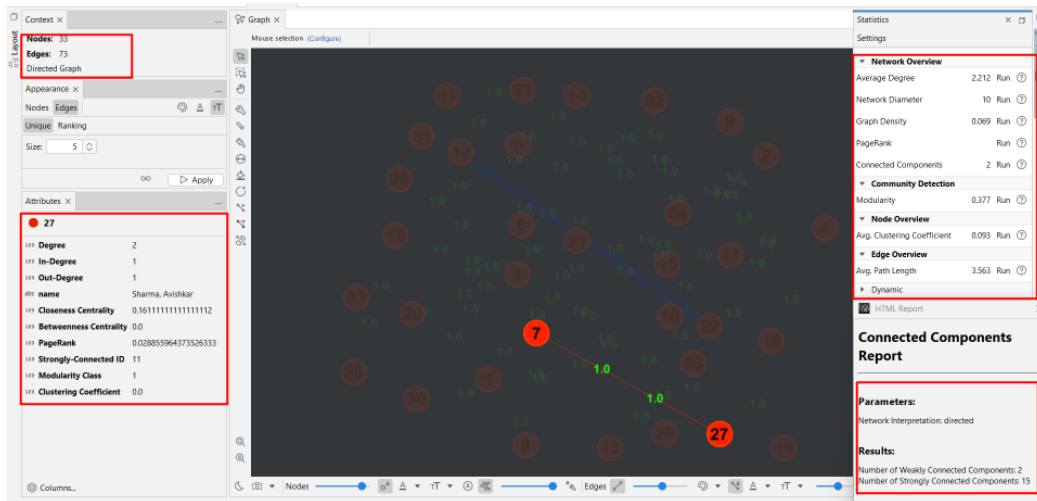


Figure 9 : My Personal Network for Contact Information

Metric Computation for my Personal Contact Information:

Metric	My Value
In-Degree	1
Out-Degree	1
Total Degree	2
Closeness Centrality	0.161
Betweenness Centrality	0.0
Page Rank	0.028
Community (Modularity Class)	1

Table 6 : My Personal Metrics for Contacts Information

Insights for Personal Metrics (Contacts Information):

With total degree of only 2 where in-degree 1, out-degree 1, I have only one contact information exchange. My betweenness of 0.0 means I do not involve as a bridge in this network, and my low closeness of 0.161 shows I cannot reach others quickly.

Eye Contact:

Property	Value
Directed / Undirected	Directed
Weighted/Unweighted	Unweighted
Justification	Eye contact can be observed unilaterally. A can have eye contact with B without letting B know. Its unweighted because data collected was binary means either a relationship exists or it doesn't.

Overall Network for Eye contact Based on Betweenness Centrality

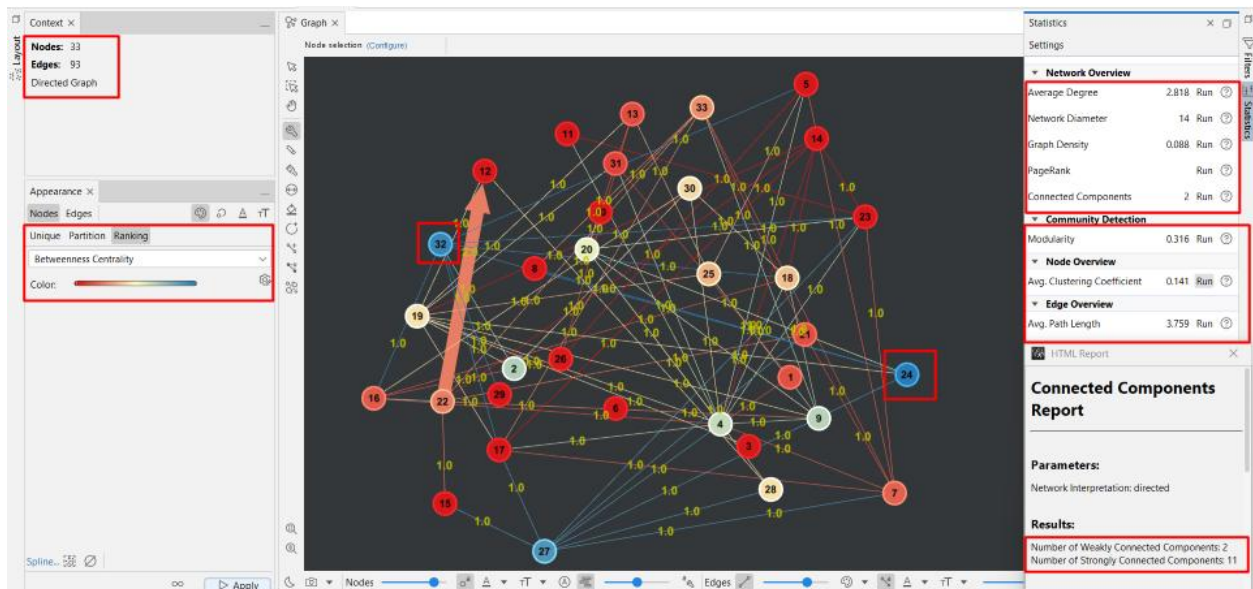


Figure 10 : Overall Network for Eye contact Based on Betweenness Centrality

Overall Network for Eye contact Based on In – Degree:

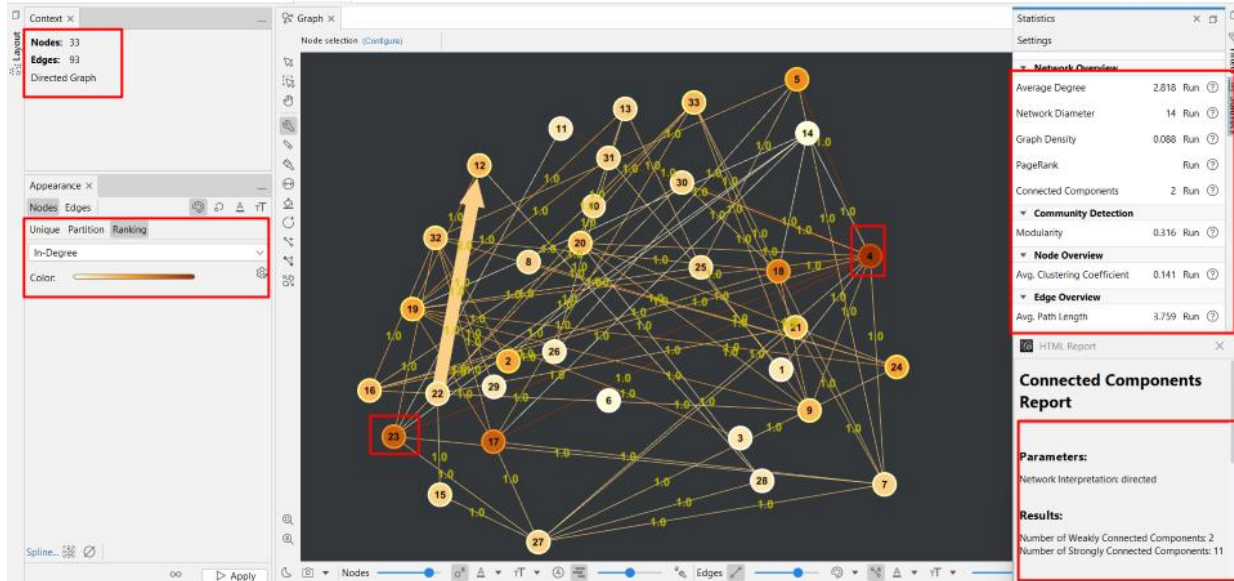


Figure 11 : Overall Network for Eye contact Based on In – Degree

Metric Computation for Overall Eye Contact:

Metric	Value
Number of Nodes	33
Number of Edges	93
Edge Density	0.088
Degree distribution	2.818
Average clustering coefficient	0.141
Number of nodes in SCC and WCC	Weakly Connected Components = 2 Strongly Connected Components = 11
Average path length (SCC)	3.759
Network Diameter	14
Top 2 By Betweenness Centrality	24 and 32
Top 2 By In-degree	23 and 4

Table 7 : Overall (Class wide) Metrics for Eye Contacts

Insights for Overall Metrics (Eye Contact):

The eye contact network appears to be sparse with a density of 0.088, which means that most students manage to make eye contact only with a few of their classmates. It has 2 Weakly Connected Components, and there is probably just one isolated group that does not share eye contact with the rest of the class. The high diameter, 14 which means a high average path length of 3.76, makes it evident that eye contact connections are highly fragmented.

The two nodes with the highest betweenness are Node 24 (Samir Rahman) and Node 32 (Lisa Trujillo). The top two in-degree nodes would comprise those receiving the most eye contact are : Node 23 (Tanvi Patil) and Node 4 (Mangesh Bhattacharya).

My personal Network for Eye Contact

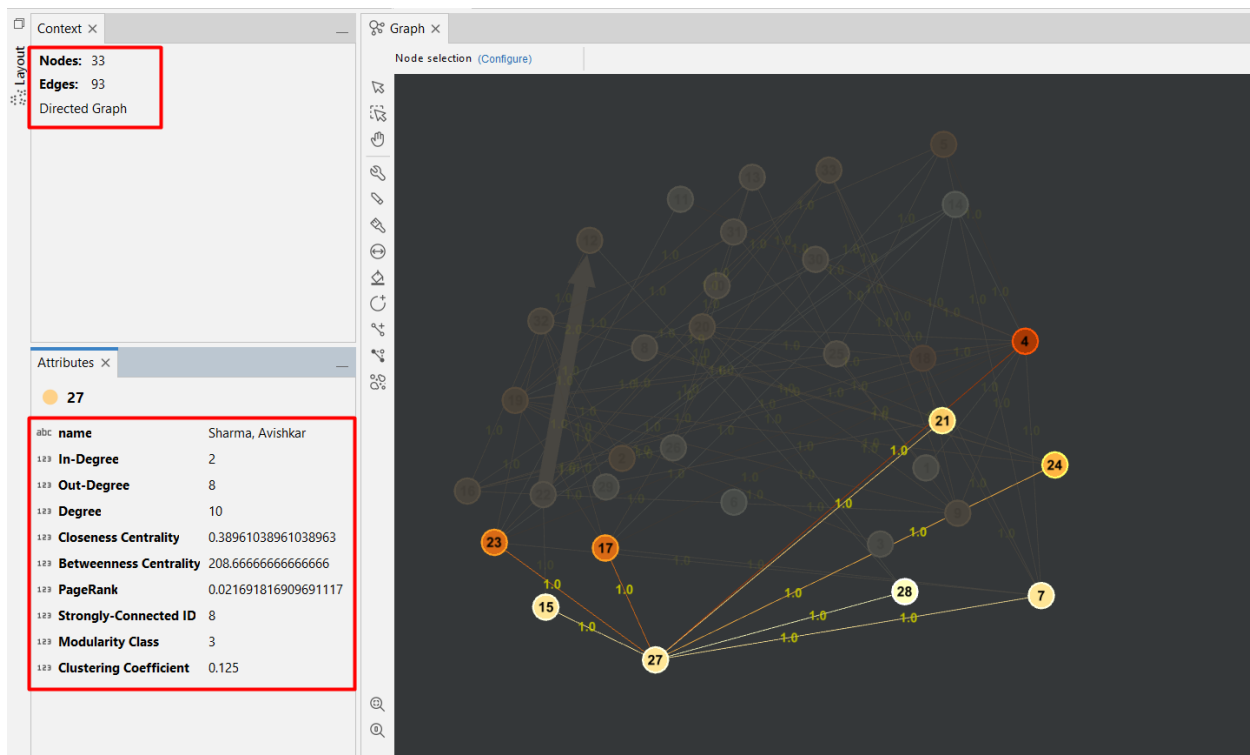


Figure 12 : My personal Network for Eye Contact

Metric Computation for my Personal Eye Contact:

Metric	My Value
In-Degree	2
Out-Degree	8
Total Degree	10
Closeness Centrality	0.389
Betweenness Centrality	208.6
Page Rank	0.021
Community (Modularity Class)	3

Table 8 : My Personal Metrics for Eye Contacts

Insights for Personal Metrics (Eye Contact):

With out-degree 8 and in-degree 2, and betweenness of 208.6 is the highest among all four networks, meaning I act as a major bridge connecting different eye contact group.

Broader Understanding of the Implications for Classroom Relationships

The four network layers demonstrate that classroom relationships may be understood differently based on type of interaction, duration, and medium. So 1-minute short conversations yield very broad and weak connections across the class: It gives effectiveness in a whole-group inclusion network since it has the highest edge count of 112 with only one weakly connected component. Longer 7-minute talks are also found to produce fewer total connections, i.e. 101 edges, and strongly locally clustered at 0.191; this suggests that in sustained interactions, students select to mutually reinforce groups of relationships. It is the most selective layer of contact information, with only 73 edges and two weakly connected components, which implies a real willingness for connection outside the classroom. The eye contact has the most diameter at 14, grasping attentional non-verbal patterns that run independently from verbal interaction.

Notable is that different students form different networks: e.g., the student with the highest degree in 1-minute conversation is 19 and 17, but those in leading betweenness centrality with eye contact are 24 and 32. This shows that no one student is centrally placed across all types of interaction. Hence interventions in the classroom are best targeted at specific relationship layers.

Cross-Network Insights emerge when examining the networks:

When the network graph taken together and examined carefully, many patterns appear that could be missed if done only individually. First, relationships have different structures. Casual conversations make a dense network, while for the sharing of contacts it tends to be very sparse and fragmented. Second, there are differing reciprocal aspects between one kind of relationship and another. For instance, mutual reporting is more common in contact than it is for sharing contact information or other. Third, core people within a social structure may shift from one network to the other. For instance, a student centrally placed in conversations may have a very peripheral location in the sharing of contact information. When these networks were analyzed one by one, patterns like betweenness, network diameter, average path cost and clustering coefficient of student's involvement would be overgeneralized. But taken together, these findings suggest that the social organization of the classroom is multidimensional, in the sense that distinctive types of relationships guide students into different places.

What Would Be Lost If the Networks Were Examined Individually

If every network is analyzed in isolation, three crucial insights are lost outright. First, the incorrect impression would be that the 1-minute and contact information networks identify the same student as being "central" within the group. While the 1-minute network says 19 and 17 have the highest degree, the contact information network shows entirely different leaders on genuine trust. Taken individually, this would be interpreted to mean that student with more 1 min connection are consequently most trusted; cross-network comparison indicates that verbal activity does not equate to relational depth.

Secondly, it is the invisible role of non-verbal influence: It is clarified in the eye contact network that students 24 and 32 have the highest betweenness centrality but reading through the 1 minute and 7-minute conversation networks shows nothing revealing them as leaders. This would have been missed by the analysis that even the quietest students receive visual attention and exert silent social influence over other students.

Thirdly, it stays in the shadow of the decomposition of real relationships. There is one weakly connected component and 112 edges in the 1-minute network, which makes the analyzed subject look like a highly connected classroom. On the other hand, the contact information network reveals two weakly connected components and only 73 edges, so the class looks less connected while being viewed through the contact information network.

Viewing this information in isolation would incorrectly suggest the class is a single large, connected group, while analysis across different networks reveals that there are deep social cleavages wherein the students who talk to everyone still refuse to share contact information. This creates invisible barriers that are only unveiled when looking across multiple layers of networks.

Cross-Network Insights Personal:

While analyzing all four personal networks I (Avishkar Sharma, ID 27) concluded that, I am most active in 1 min communication within degree of 2 and out degree of 8. I am minimally involved in contact information sharing with total degree of 2. I have the highest eye contact of 206.6, means that I play a bridging role in visual communication. In the case of contact information my betweenness drops to 0 which indicates my no bridging role. The diameter varies across networks ranging from 7 to 14, showing that the relationship has different level of connectivity. If I had just analyzed the whole network I would miss these role differences, like how I can be a bridge in one context but isolated in another.

Question Number 2:

For the Assignment I was asked to create a working Web Crawler for ontariotechu.ca and uoit.ca.

The main objection of the crawler is to crawl 10,000 HTML pages and build network graphs.

Libraries used:

Library	Purpose
requests	To download webpages via HTTP GET
BeautifulSoup	To parse HTML and extract <a href> (hyperlink) tags
urllib.parse	Urllib is used to convert relative link
networkx	To build the directed graph of crawled pages
time	I used time to add delays such that to avoid from being blocked from website
collections.deque	This is used for FIFO queue because use BFS and BFS uses FIFO (First In First Out)
os	To create folders and save the output files

Table 9 : List of Library Used

High level flow-chart of code:

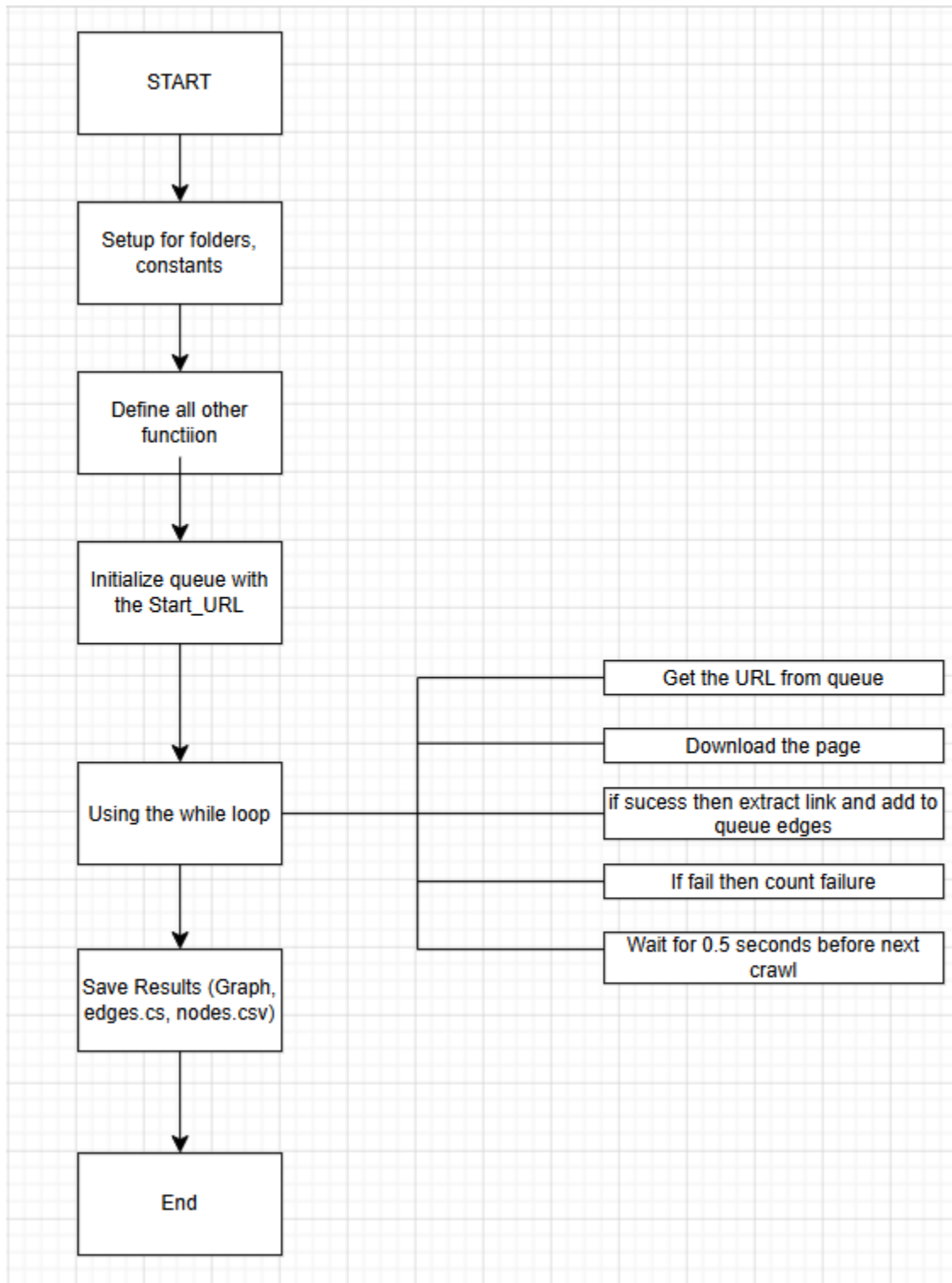


Figure 13: High level flow chart of the code

Full Source Code for crawler :

Web Crawler Code:

Note: I did this code with the help of an AI assistant for coding. I have personally reviewed and understood every line. I traced the entire program's flow and went through libraries like requests, BeautifulSoup, and pandas. It's the AI that gave me the syntax and structure; however, the understanding and debugging are purely mine.

requests send the HTTP GET requests to a URL and returns the HTML content. We need this to download webpages.

```
import requests
```

BeautifulSoup is used to make easy to find all href tags in order to parse HTML and extract hyperlinks

```
from bs4 import BeautifulSoup
```

I also imported urljoin and urlparse to convert relative links like /about to absolute URL and to break the URL into parts i.e. protocol, domain and path to verify if it is internal or not.

```
from urllib.parse import urljoin, urlparse
```

This is only used to build and store the graph of all crawled websites.

```
import networkx as nx
```

```
import time
```

Since I will be doing BFS so deque provides efficient pop from left.

```
from collections import deque
```

To create folders and save files.

```
import os
```

```

#To create a folder name ontariotech_crawl and save the files into it.
SAVE_FOLDER = r"D:\complex_network\ontariotech_crawl"
#This will show the folder where it saved files.
print(f"Files will be saved to: {SAVE_FOLDER}")
#Here, I used total of 10,000 fetches with 0.5 seconds between requests and started from
https://ontariotechu.ca/ for crawling.
MAX_ATTEMPTS = 10000
DELAY = 0.5
START_URL = "https://ontariotechu.ca/"

# =====
# DATA STORAGE
# =====

#This will store URLs that are crawled
visited = set()

#This will create a queue of websites in FIFO order to be crawled.
queue = deque()

#This will give the list of edges its the connection between the main page and its sub pages.#
edges = []

#These are used to check the number of pages downloaded and number of pages failed requests.
Total will calculate success and failed one.#
success_count = 0
failed_count = 0
attempt_count = 0

```

#This will check if the url is coming from ontariotechu.ca or uoit.ca. If its from either from two it returns true else false for external links.#

```
def is_internal_link(url):
```

```
    parsed = urlparse(url)
```

```
    if "ontariotechu.ca" in parsed.netloc or "uoit.ca" in parsed.netloc:
```

```
        return True
```

```
    return False
```

#It will filter out non-HTML links that cannot be crawled. This will skip empty URLs, javascript and media files. It will return true only when it crawl for HTML pages. The loop iterates through the sjiip list and it uses url.lower().endswith(ext) to verify if the url ends with unwanted extension. Only if the URL passes all checks of not empty, not javascript and not a media file then only it returns true, meaning it is a valid HTML page to be crawled.

```
def should_crawl(url):
```

```
    if not url:
```

```
        return False
```

```
    if url.startswith('#') or url.startswith('javascript:')
```

```
        return False
```

```
    skip = ['.pdf', '.jpg', '.png', '.gif', '.mp4', '.mp3', '.zip', '.css', '.js']
```

```
    for ext in skip:
```

```
        if url.lower().endswith(ext):
```

```
            return False
```

```
    return True
```

Here I track number of failed requests. This function tracks the number of failed request using a global failed_count variable. It downloads a webpage using an HTTP GET request. User-Agent that acts like a real browser say Chrome or Firefox, is used to avoid from being blocked and returning the ocontent only if the HTTP status code is 200(OK); else it increments the failed count. Only 200 because its it only code that will be returned if the page is running properly.

```
def fetch_page(url):
```



```

global failed_count

try:

    headers = {'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36'}

    response = requests.get(url, timeout=10, headers=headers)

    if response.status_code == 200:

        return response.text

    else:

        failed_count += 1

        return None

except:

    failed_count += 1

    return None

```

Here the function `extract_links` is used to parse the HTML content of a webpage using BeautifulSoup class to find all anchor tags. It creates relative URLs to absolute using the URL join to every link found. For each of the URL it uses `strip()` to remove any leading or trailing space from href value, so that we get only the clean URLs. If not, the `should_crawl(href)` statement checks the URL if it should crawl or not, if it is False meaning the link is a pdf image or media and the `continue` means moves to the next link. It also removes the fragment identifier, since those points to section within the same page. Then we have `should_crawl` to filter out non-HTML files and `is_internal_link` is used to keep the URLs belonging to `ontariotechu.ca` or `ouit.ca`. Only the valid links are added to a list, and we returned them at the end. This is done to collect the internal, crawlable HTML pages from the website.

```

def extract_links(html, base_url):

    soup = BeautifulSoup(html, 'html.parser')

    links = []

    for a in soup.find_all('a', href=True):

        href = a['href'].strip()

        if not should_crawl(href):

```

```

        continue

    full_url = urljoin(base_url, href)
    full_url = full_url.split('#')[0]
    if is_internal_link(full_url):
        links.append(full_url)

    return links

# =====

# MAIN CRAWLER

# =====

#I used visual separator "="*60 to make output easier to read clearly and separate each sections
print("="*60)

#This will ensure that the program is running before any crawling happens.
print("WEB CRAWLER STARTED")

#This is to show that I run the crawler.
print(f"Author: Avishkar Sharma")

#Shows where crawling begins, starting from which
print(f"Starting URL: {START_URL}")

#This is to display when the crawling stops so that I can know it ended.
print(f"Max attempts: {MAX_ATTEMPTS}")

#I used visual sepataor "="*60 to make output easier to read clearly.

```

```
print("="*60)
```

#A deque is a double ended queue that stores URLs waiting to be crawled and the append() will add the URL to the right end of the queue using FIFO (First In First Out) and the starting URL is a homepage of Ontario to the queue so BFS has a first page to crawl.#

```
queue.append(START_URL)
```

#The while loop will be executing as long as there are available URLs in the queue and till we don't reach the maximum attempts of 10,000 crawls.

```
while queue and attempt_count < MAX_ATTEMPTS:
```

#Popleft() removes and returns the URL from the front of the queue in FIFO order, so that the pages discovered earlier are crawled first.

```
    current_url = queue.popleft()
```

#This is to skip if we have already crawled this URL to prevent duplicate processing.

```
    if current_url in visited:
```

```
        continue
```

#This is to increase the attempt counter and calculate the percentage of completion

```
    attempt_count += 1
```

```
    percent = (attempt_count / MAX_ATTEMPTS) * 100
```

#This is for the progress and download page, it prints current progress with attempt number and percentage with 1 floating point number.

```
    print(f"\n[{attempt_count}/{MAX_ATTEMPTS}] ({percent:.1f}%)")
```

#It will print the URL being crawled but only shows the first 80 characters and the following ... indicate the URL was cut off because it was too long.

```

print(f"Crawling: {current_url[:80]}...")

#Here we download the webpage and this will return HTML if success and none if failed.
html = fetch_page(current_url)

#Now we check if the page was downloaded successfully
    if html:
#Sucess Case:
# Increment success counter by 1
success_count += 1

#This will add current URL to visited set so we dont have duplication
visited.add(current_url)

#Now extracting all internal links from the HTML content
links = extract_links(html, current_url)

#Print sucess message showing how many links were found on this page
    print(f" SUCCESS! Found {len(links)} links")

#This will print the total number of unique pages crawled so far
print(f" Total pages so far: {success_count}")

#Running the loop through each of the link found on the page
#edges.append will add a directed edge from current page to linked page i.e from source to target
    for link in links:

```

```

edges.append((current_url, link))

#Now checking the link that its not visited and not already in queue then adding at the end of
current queue

    if link not in visited and link not in queue:

        queue.append(link)

    else:

        #Incase of failure means that the page could not be downloaded

        print(f" FAILED")

# Pause the crawler to be respectful to the server which will prevent from getting blocked
time.sleep(DELAY)

# =====

# SAVE RESULTS

# =====

#I used a visual separator with blank for clean display
print("\n" + "="*60)

#To announce that crawling has finished
print("CRAWLING COMPLETE")

#I used a visual separator with blank for clean display
print("="*60)

#This will display the total number of attempts i.e success and failed

```

```

print(f"Total attempts: {attempt_count}")

#This will display the total number of pages successfully downloaded
print(f"Successful pages: {success_count}")

#To display the number of failed attempts with 404 or timeout or connection errors
print(f"Failed pages: {failed_count}")

#This will display the total number of hyperlinks discovered across all pages
print(f"Total links: {len(edges)}")

#Now calculating and displaying success rate as percentage and formatted to 1 decimal place
print(f"Success rate: {(success_count/attempt_count)*100:.1f}%")

# Build graph
# To create an empty directed graph. Edges have direction: from the source page has link to
target page.
G = nx.DiGraph()

#All the crawled hrperlinks or edges, and all of edge connects two pages i.e from current_url to
linked url
G.add_edges_from(edges)

# Save files

#Now I will save the graph to a file using gexf format format for gephi format, using this format
GEXF , can be opened directly without conversion.
nx.write_gexf(G, os.path.join(SAVE_FOLDER, "ontariotech_graph.gexf"))

```

#Showing the file was saved in the disk

```
print(f"\n Graph saved")
```

#Here, I created a CSV file name edges.csv inside the SAVE_Folder, where w mode will allow to write in the file and encoding ='utf-8' ensures special characters are saved correctly and as f assigns the file object to variable 'f' for use inside the block.

with open(os.path.join(SAVE_FOLDER, "edges.csv"), 'w', encoding='utf-8') as f:

#now writing the header row and the column name for target, where source is the page with the link(node) and target is the page being linked.

```
f.write("source,target\n")
```

#Running the loop through each edge stored in the edges list, src is the source URL and tft is the target URL

```
for src, tgt in edges:
```

#This will ensure only one row per edge and double quotes around URLs handle commas or special character that could be present in the edge and /n will create new line after each edge

```
f.write(f"{src}","{tgt}"\n')
```

#After the block ends, the file will be closed on its won, and it prints confirmation message showing how many edges were saved.

```
print(f' Edge list saved ( {len(edges)} edges)')
```

This will create a nodes.csv with utf-8 encoding and it will write header row : id and url.

#enumerate() assigns automatic numeric IDs to each of the webpages and each url is enclosed in double quotes to handle special characters safely.

with open(os.path.join(SAVE_FOLDER, "nodes.csv"), 'w', encoding='utf-8') as f:

```
f.write("id,url\n")
```

```
for i, url in enumerate(visited):
```

```
f.write(f'{i},{url}\n')
```

```
#Printing the confirmation with total number of node count
```

```
print(f' Node list saved ({len(visited)} nodes)')
```

```
#This will create a summary.txt in the file location, as w mode is enabled it could write and it  
uses system default text.
```

```
with open(os.path.join(SAVE_FOLDER, "summary.txt"), 'w') as f:
```

```
#It will write my name on the first line and the creates a new line after writing
```

```
f.write(f'Author: Avishkar Sharma\n')
```

```
#Now writing the current date and time to check the time of crawling and this is done by  
time.strftime() formats the timestamp as Year-Month-Day Hour:Minute:Second
```

```
f.write(f'Date: {time.strftime('%Y-%m-%d %H:%M:%S')}\n')
```

```
#Adding a blank line to separate header from data
```

```
f.write(f'\n')
```

```
#Writing the total number of fetch attempts, it includes both success and failed
```

```
f.write(f'Attempts: {attempt_count}\n')
```

```
#Writing the number of successfully downloaded HTTP status pages
```

```
f.write(f'Success: {success_count}\n')
```



```

#Writing the number of failed requests with 404 or timeout or connection errors
f.write(f'Failed: {failed_count}\n')

#Write total number of unique hyperlinks or edges identified
f.write(f'Edges: {len(edges)}\n')

#Write total number of unique webpages or nodes identified
f.write(f'Nodes: {len(visited)}\n')

#Calculating the success rate as percentage given by success/total*100 and .1f formats to 1
decimal place
f.write(f'Success rate: {(success_count/attempt_count)*100:.1f}%\n')

#This will print the confirmation message indicating summary file was saved
print(f'Summary saved')

#Print a blank line followed by "="*40 for a visual separator.
print("\n" + "="*40)

#Printing success message showing everything done completely!
print("ALL DONE!")

#This will print the folder location where all files were saved.
print(f'Files saved in: {SAVE_FOLDER}')

# Printing a visual separator of 40 to mark the end of output.
print("="*40)

```

Metrics to Report:

Metrics	Value
Number of Nodes	24935
Number of Edges	1975474
Edge Density	0.003
Average clustering coefficient	0.002
Strongly Connected Component (SCC)	16594
Weakly Connected Component (WCC)	1
Average path length (SCC and WCC)	3.783
Diameter (SCC and WCC)	6
Community Detection (Modularity)	0.712

Table 10 : All the Metrics

Degree Distribution:

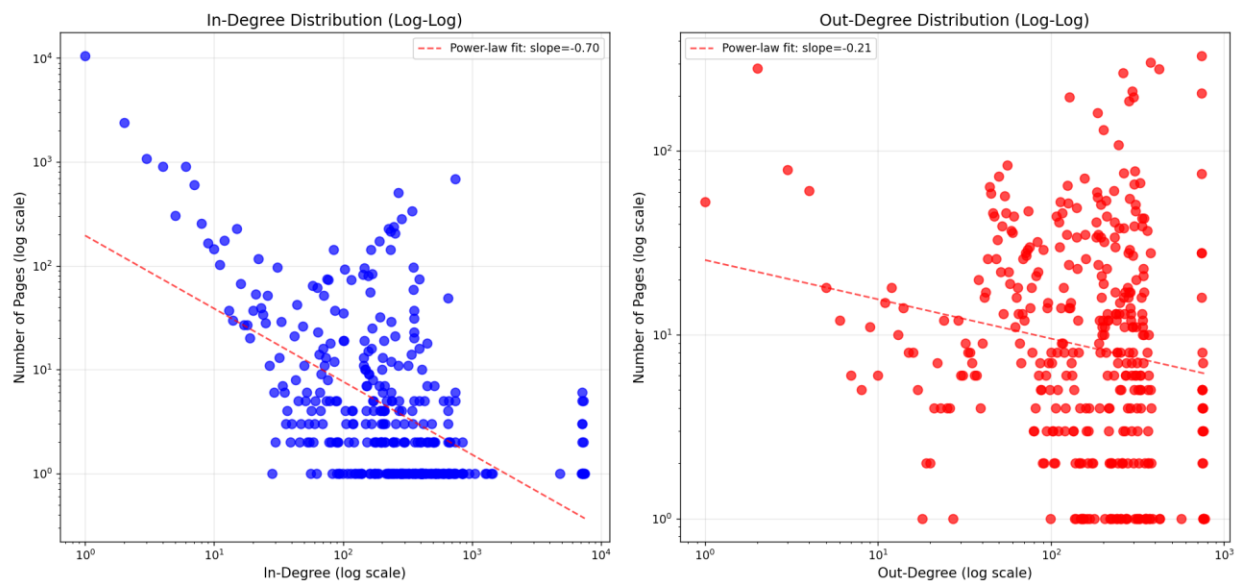


Figure 14: Degree Distribution based on log-scale

Connected Component Report:

Parameters:

Network Interpretation: directed

Results:

Number of Weakly Connected Components: 1

Number of Strongly Connected Components: 16594

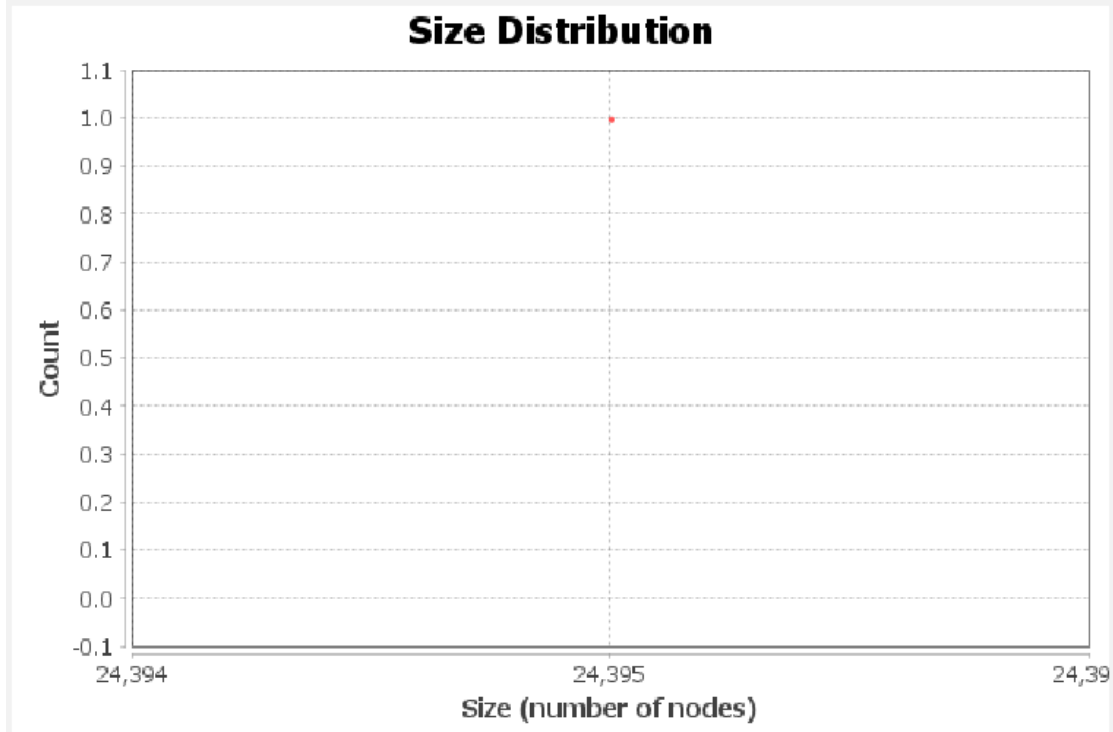


Figure 15 : Number of WCC and SCC

Closeness Centrality Distribution:

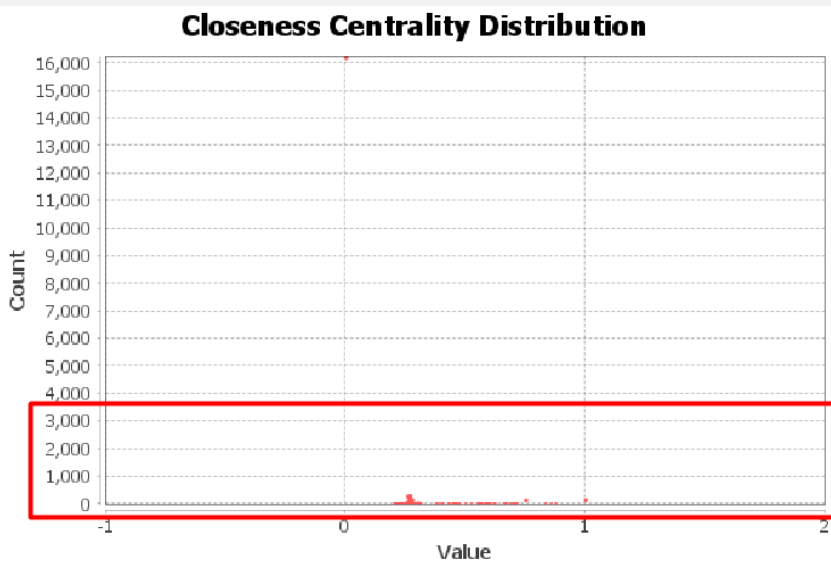


Figure 16 : Closeness Centrality Distribution

Betweenness Centrality Distribution:

Parameters:

Network Interpretation: directed

Results:

Diameter: 6

Radius: 0

Average Path length: 3.782745166765004

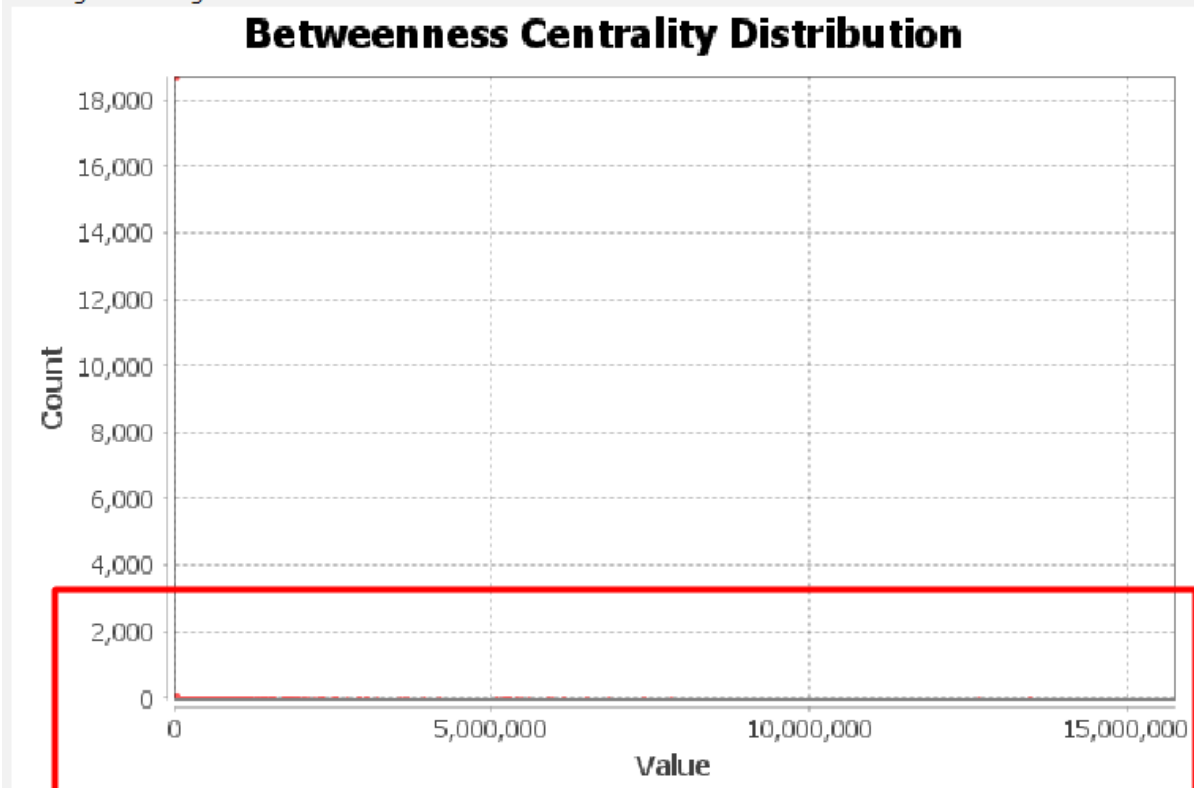


Figure 17 : Betweenness Centrality Distribution

Page Rank Distribution:

Parameters:

Epsilon = 0.001
Probability = 0.85

Results:

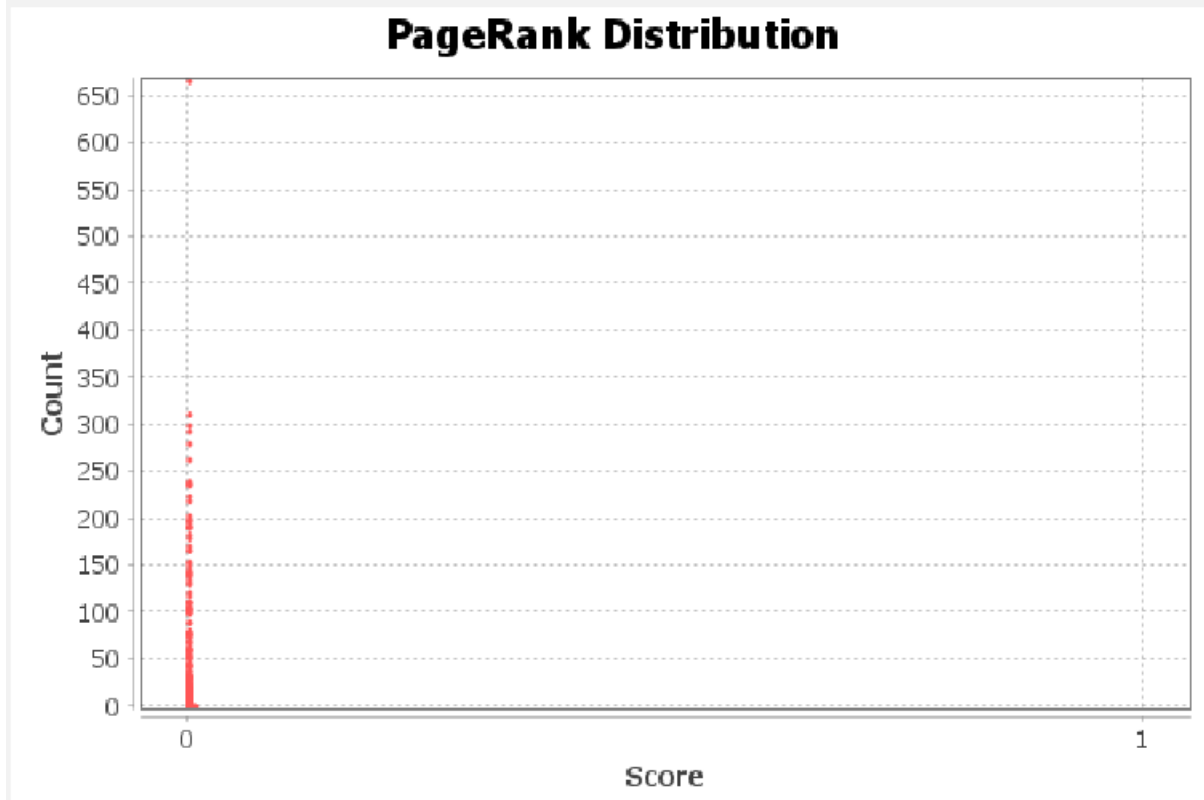


Figure 18 : Page Rank Distribution

Analysis for using Excel over Gephi:

The process moved consistently and systematically from the completed web crawling through the stages of the network analysis workflow. The first step involved loading raw output files produced by the crawler: nodes.csv which contains 24,935 node IDs along with URLs and edges.csv with containing 1,975,474 source-target pair into Python using the panda's library.

The second step was to attempt calculation of the centrality measures in Gephi, however I was facing continuous capacity issue frequently because of very large graphs and was crashing due to limited memory availability with performance limitations. In some cases when processing almost 2 million edges in the graph. So I had to resort to my other approach for the CSV-based method and used Microsoft Excel to make some calculations. To avoid depending entirely on Gephi for the degree distribution analysis, I exported the edge list into the CSV format and took the pivot table feature of Excel to count incoming and outgoing links for each unique URL. This way was

more stable, and I could identify the top 10 and bottom 10 pages quickly for both in-degree and out-degree measures.

Third, I did those metrics that involved graph algorithms: connected components, path length, diameter, clustering coefficient, and modularity, again in Gephi because it has been optimized for such computations. The full Python code that loads CSV files and calculates degrees components using the counter class of collections, then derives final centrality results, can be found in the Appendix section. This hybrid methodical tactic to Excel for simple counting tasks and Gephi for graph algorithms at its back allowed me to compute every metric accurately without losing data or experiencing crashes in the software.

Top 10 Most and Least central Webpages:

Top 10 Highest In-Degree (Most Popular pages):

S.N	URL	Incoming Links
1	https://ontariotechu.ca/sites/library/	7496
2	https://accessibility.ontariotechu.ca	7374
3	https://ontariotechu.ca/future-students/undergraduate/campus-tours-and-events/index.php	7369
4	https://ontariotechu.ca/apply	7334
5	https://ontariotechu.ca/current-students/academic-calendars/index.php	7334
6	https://accessibility.ontariotechu.ca/essential-accessibility/index.php	7334
7	https://ontariotechu.ca/bookstore	7334
8	https://itsc.ontariotechu.ca	7334
9	https://ontariotechu.ca/canvas/catalog-login/	7332
10	https://ontariotechu.ca/campus-services/service-disruptions/emergency-information.php	7332

Table 11 : Most Popular Pages

Top 10 Lowest In-Degree (Least Popular pages)

S.N	URL	Incoming Links
1	https://ontariotechu.ca/about/campus-buildings/north-oshawa/campus-recreation-and-wellness-centre.php	1
2	https://usgc.ontariotechu.ca/-archived-governance/contact-us/index.php	1
3	https://usgc.ontariotechu.ca/-archived-governance/academic-council/index.php	1
4	https://usgc.ontariotechu.ca/-archived-governance/appeals/index.php	1
5	https://ontariotechu.ca/programs/undergraduate/health-sciences-and-medicine/faculty-of-health-sciences-pre-med.ph	1
6	https://ontariotechu.ca/sites/library/services/index-old	1
7	https://usgc.ontariotechu.ca/-archived-governance/board-of-governors/resources.php	1
8	https://sites.uoit.ca/openhouse/index.php	1

9	https://ontariotechu.ca/programs/undergraduate/health-sciences-and-medicine/faculty-of-science-pre-med.php	1
10	https://gradstudies.ontariotechu.ca/grad-student-engagement/base-camp/base-camp-achieve/grad-pro-skills/partners.php	1

Table 12 : Least Popular Pages

Top 10 Highest Out-Degree (Most Linking Pages):

S.N	URL	Outgoing Links
1	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/index.php	753
2	https://studentvoices.ontariotechu.ca/quizzes/index.php	762
3	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-09.22.2025.php	759
4	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-09.08.25.php	758
5	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-10.21.2024.php	757
6	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-01.27.2025.php	757
7	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-05.05.2025.php	757
8	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-10.7.24.php	756
9	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-11.04.24.php	756
10	https://studentvoices.ontariotechu.ca/ridgeback-report/ridgeback-report-past-issues/ridgeback-report-06.02.2025.php	756

Table 13 : Most Linking Pages

Top 10 Lowest Out-Degree (Least Linking Pages):

S.N	URL	Outgoing Links
1	https://ontariotechu.ca/about/campus-map.php	0
2	https://ontariotechu.ca/about/leadership-and-governance/-archived-office-of-the-president/strategicplanning/index.php	0
3	https://gradstudies.ontariotechu.ca/current_students/_new_students/communicating-with-you.php	0
4	https://ontariotechu.ca/current-students/campus-services/index.php	0
5	https://bookaspace.ontariotechu.ca/portal/	0
6	https://ontariotechu.ca/partners/index.php	0
7	https://ontariotechu.ca/research/index.php	0
8	https://ontariotechu.ca/giving/index.php	0
9	https://research.ontariotechu.ca/discover-research/iaea-collaborating-centre/archived-ocni-cnrc-march2026/index.php	0

10	https://research.ontariotechu.ca/faculty/intellectual-property/ip-education-resources.php	0
----	---	---

Table 14 : Least Linking Pages

Interpretation:

The observed network graph from crawling the webpage is a directed and unweighted graph. It is directed because hyperlinks have directions from a source page to a target page, for example a link from A to page B but it does not mean a link goes back from page B to page A. It is unweighted because each of the link or edges are treated equally without assigning any important score.

The network comprises 24,935 nodes and 1,975,474 edges, representing linkages between all web pages under the domain names ontariotechu.ca and uoit.ca. Edge density is 0.003; hence, a mere 0.3% of all potential connections are present. The least connected graph is structurally very sparse. A full graph would include over 310 million links, but these would not be useful or meaningful for a university website. Low density implies that pages selectively link to only their relevant content, another reason one does not link everywhere.

Node Count Discrepancy:

Gephi reports a count of 24,935 nodes representing that Gephi file; however, the nodes.csv file uncovers somewhat about 8,000 nodes. This discrepancy arrives from the complete graph, being stored in the Gephi file, including all nodes found during crawling compared against the nodes.csv file, which contains nodes having at least one incoming or outgoing edge. The remaining 16,688 nodes, around 67 percent of the total, isolated pages found through sitemaps or external references without hyperlink connections within the crawled dataset, are those types of pages. This includes orphan pages, pages with crawling errors, redirected URLs, and sitemaps only. All the analysis was done on that Gephi graph, such that the full 24,935 nodes represent the full crawled dataset.

Degree Distribution and Page Centrality

In-degree is that of the number of links pointing to a page; it gives a good popularity and importance of that page. The most incoming links is the library site with ending in /sites/library, with 7,496. The following are the accessibility site, with 7,374, and campus tours and events, with 7,369. The remaining top 10 pages are quite stable in the range between 7,300 and 7,500 incoming links.

The least popular pages (non-zero in-degree) have just one incoming link. Mainly archives and old event pages (archived-in the URL), specialized pages that have only one link from which they stem. The low in-degree strongly suggests that this is outdated material, no longer actively pushed serves a specific audience. Pages with zero in-degree are much more numerous and contain fully orphaned information, as there are no references from any other pages.

The out-degree tells how many links are present on a page, and that really indicates whether the page is acting as a sort of pivotal hub for navigation. All top out-of-degree pages are from the student newspaper. The quiz page has 762 out-links; past issues and their various subpages will all be found in the 756 - 0759 range.

Pages with the lowest out-degree excluding zero are included landing pages, PDF viewers, document repositories, and login pages that redirect without basing routing upon the hyperlink

structure. Zero out links do represent dead ends to the navigation from the page; thus, after arriving at such a page, a user has no hyperlinks to continue from.

Average Clustering Coefficient:

Given that average clustering is only 0.002. The clustering coefficient looks at how many of a page's neighbors in a network are connected to one another. The interpretation of this value is such that a value of '1' would mean every pair of neighbors was connected, and a value of '0' would mean no neighbour pairs were connected. A value at 0.002 means that there does not seem to be any small-world properties in the ontariotechu.ca network. Pages link mainly to parent and child pages rather than making triangles of relations with each other. This is something people generally find with organizational websites, where content follows a clear information hierarchy rather than a web of reference densely coupled with links.

Connected Components Analysis:

Strongly Connected Components (SCC): SCC is defined as a set of pages in which the user can travel from any page to any other page following the direction of links. 16,594 SCCs are found in this network, which clearly signifies that the directed graph is highly fragmented. Of most of these components, size 1 is holding back, meaning again that they are such pages that cannot be reached from any other page by following link direction. Most of the nodes are present in the largest SCC, and the bidirectional navigability is, therefore, relatively rare. This is as one would expect: with most web links being unidirectional, very few pairs of pages link back to one another reciprocally.

Weakly Connected Components (WCC): WCC is computed based on the graph with link directions ignored; it is quite different from the WCC obtained when we consider links traveling in opposite directions with different weights. As per the network, there is 1 WCC, which tells us that all 24,935 nodes are connected if one disregards the link direction. This is important because, although a very tiny portion of links points in both directions, it still connects the entire website into one single connected component under an undirected graph. In other words, a user will be able to reach any one page from any other page so long as the proper links are in place—without caring for the direction in which they are following, even if sometimes they must go "backward" relative to the link's direction. This single big component, emerging out of 100% of the nodes, obviously says that a website's general connectivity is excellent at the lowest cost.

Average Path Length and Diameter

Average path length is 3.783; thus, pages within an average path of two in the giant connected component are 4 clicks apart. This is a very low number even for a website with close to 25,000 pages. This is once again a clear application of the "six degrees of separation" principle: the user, therefore, reaches any page in four clicks on average. This clearly points to very good information architecture, where the key pages are not too deep in the hierarchy.

The observed diameter is 6; it means the maximum shortest path from any pair of pages is going to take at most 6 clicks. These again come under the rare category, and in general users rarely would need more than 6 navigation steps to get to the content on a site. Combine this with the average path length of 3.78, and one gets a very effective presentation of navigation to the ontariotechu.ca website with a shallow organization of content.

Conclusion:

One inference drawn could be that the analysis of the website network of ontariotechu.ca members show a well-structured and easily navigable site giving a clear organization by topics. In detail, this website comprises 24,935 pages with almost 2 million hyperlinks, yet the average path length is less than four clicks, and the diameter is only six clicks. The very good modularity score of 0.712 justifies that the content is segmented into meaningful communities logically based on the organization of the university.

It contains a library, accessibility, admissions, and academic calendars in which more than 7,000 incoming links have been connected to each of them; these pages are exceedingly important for every user. There is only one weakly connected component with all nodes, showing that the whole link structure of the website is connected; meaning no part of the content is isolated. This is obvious to have a hierarchical decomposition of 16,594 SCCs, as most of the links on the Web are one way and not reciprocal. In essence, the site ontariotechu.ca is a splendid model of a university web architecture one that comes complete with easily navigable, distinct topical clusters of information and connectivity.

Visualization of web Crawled using Gephi:

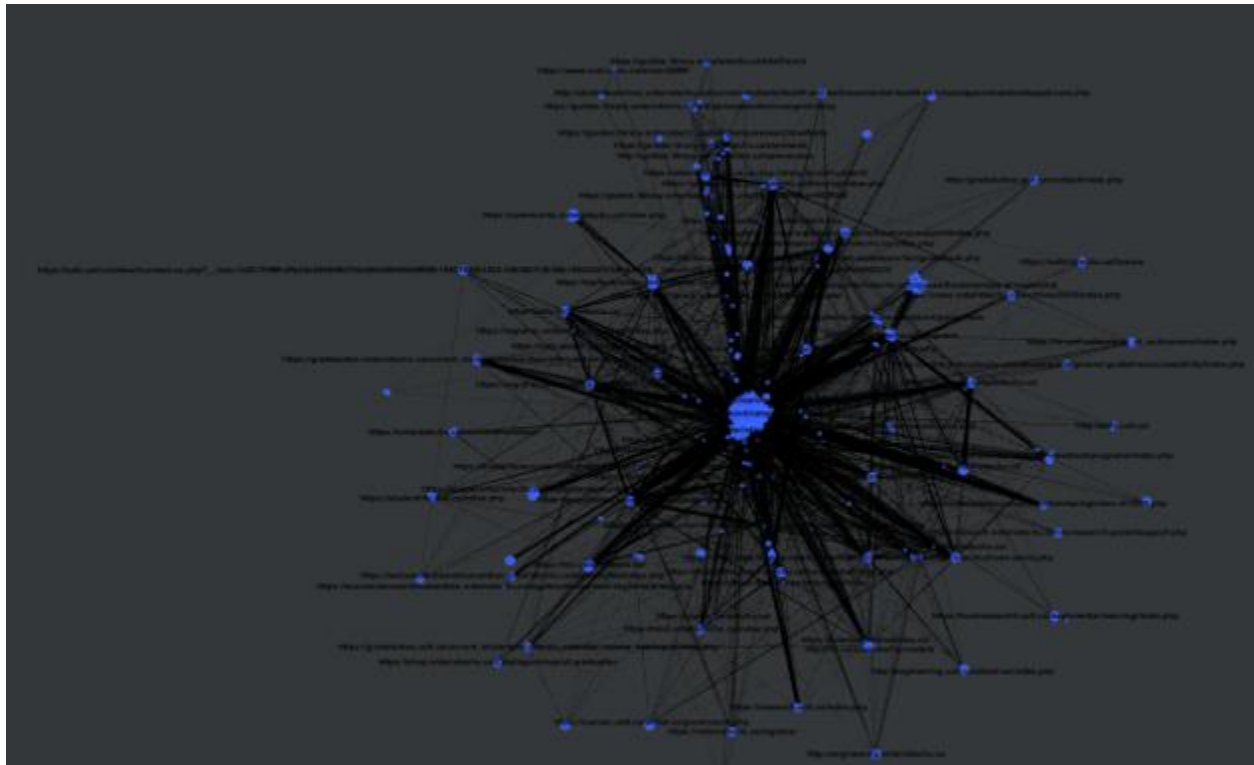


Figure 19 : Web Crawler Representation

Appendix:

Code used to identify top 10 most and least central pages.

Note, I did this code with the help of an AI assistant for coding. I have personally reviewed and understood every line. I traced the entire program's flow and went through libraries like counter, time, and pandas. It's the AI that gave me the syntax and structure; however, the understanding and debugging are purely mine.

```
import pandas as pd

from collections import Counter

import time


print("="*70)
print("CENTRALITY ANALYSIS - CORRECTED VERSION")
print("="*70)


start = time.time()


# =====
# STEP 1: Load files
# =====

print("\n Loading files...")


nodes_df = pd.read_csv(r"D:\complex_network\ontariotech_crawl\nodes.csv")
edges_df = pd.read_csv(r"D:\complex_network\ontariotech_crawl\edges.csv")


print(f" Nodes: {len(nodes_df):,}")
print(f" Edges: {len(edges_df):,}")


# =====
# STEP 2: Convert everything to STRING to avoid type issues
# =====
```

```

print("\n Processing data...")

# Convert node IDs to string and create mapping
nodes_df['id_str'] = nodes_df['id'].astype(str)
id_to_url = dict(zip(nodes_df['id_str'], nodes_df['url']))

# Convert edge columns to string
edges_df['source_str'] = edges_df['source'].astype(str)
edges_df['target_str'] = edges_df['target'].astype(str)

# =====
# STEP 3: Count degrees
# =====

in_degree = Counter() # Links TO a page
out_degree = Counter() # Links FROM a page

# Count edges
for idx, row in edges_df.iterrows():
    source = row['source_str']
    target = row['target_str']

    # Count outgoing from source
    out_degree[source] += 1
    # Count incoming to target
    in_degree[target] += 1

# =====
# STEP 4: Map back to URLs
# =====

in_degree_url = {}

```

```

out_degree_url = {}

for node_id_str, url in id_to_url.items():
    in_degree_url[url] = in_degree.get(node_id_str, 0)
    out_degree_url[url] = out_degree.get(node_id_str, 0)

# =====
# STEP 5: Verify we have data
# =====

total_in = sum(in_degree_url.values())
total_out = sum(out_degree_url.values())

print(f"\n Total incoming links counted: {total_in:,}")
print(f" Total outgoing links counted: {total_out:,}")
print(f" Pages with >0 incoming: {sum(1 for c in in_degree_url.values() if c > 0):,}")
print(f" Pages with >0 outgoing: {sum(1 for c in out_degree_url.values() if c > 0):,}")

if total_in == 0:
    print("\n ERROR: No links counted!")
    print("  Showing first few rows for debugging:")
    print("\n  First 3 nodes:")
    print(nodes_df[['id', 'url']].head(3))
    print("\n  First 3 edges:")
    print(edges_df[['source', 'target']].head(3))

    print("\n" + "="*70)
    print("SCRIPT COMPLETE (WITH ERROR)")
    print("="*70)
    print("Window will close automatically in 2 minutes...")
    print("Or press any key to close now.")

```

```

try:
    time.sleep(120)
except KeyboardInterrupt:
    pass
exit()

# =====
# STEP 6: TOP 10 HIGHEST IN-DEGREE
# =====

print("\n" + "="*70)
print("TOP 10 MOST CENTRAL PAGES (Most Incoming Links)")
print("="*70)

# Get pages with at least 1 incoming link
pages_with_links = [(url, count) for url, count in in_degree_url.items() if count > 0]
top_in = sorted(pages_with_links, key=lambda x: x[1], reverse=True)[:10]

for i, (url, count) in enumerate(top_in, 1):
    # Simplify URL for display
    if 'ontariotechu.ca' in url:
        display = url.split('ontariotechu.ca')[1] if url.split('ontariotechu.ca')[1] else '/' (HOMEPAGE)'
    elif 'uoit.ca' in url:
        display = url.split('uoit.ca')[1] if url.split('uoit.ca')[1] else '/' (HOMEPAGE)'
    else:
        display = url[:80]
    print(f"{i}. {display}")
    print(f"    {count:,} incoming links\n")

# =====

```

```

# STEP 7: TOP 10 LOWEST IN-DEGREE (but >0)
# =====

print("="*70)
print("TOP 10 LEAST CENTRAL PAGES (Fewest Incoming Links - but at least 1)")
print("="*70)

least_in = sorted(pages_with_links, key=lambda x: x[1])[:10]

for i, (url, count) in enumerate(least_in, 1):
    if 'ontariotechu.ca' in url:
        display = url.split('ontariotechu.ca')[1] if url.split('ontariotechu.ca')[1] else '/'
    elif 'uoit.ca' in url:
        display = url.split('uoit.ca')[1] if url.split('uoit.ca')[1] else '/'
    else:
        display = url[:80]
    print(f'{i}. {display}')
    print(f'{count} incoming links\n')

# =====

# STEP 8: TOP 10 HIGHEST OUT-DEGREE
# =====

print("="*70)
print("🔗 TOP 10 MOST CENTRAL PAGES (Most Outgoing Links)")
print("="*70)

pages_with_out = [(url, count) for url, count in out_degree_url.items() if count > 0]
top_out = sorted(pages_with_out, key=lambda x: x[1], reverse=True)[:10]

for i, (url, count) in enumerate(top_out, 1):

```



```

if 'ontariotechu.ca' in url:
    display = url.split('ontariotechu.ca')[1] if url.split('ontariotechu.ca')[1] else '/' (HOMEPAGE)
elif 'uoit.ca' in url:
    display = url.split('uoit.ca')[1] if url.split('uoit.ca')[1] else '/' (HOMEPAGE)
else:
    display = url[:80]
print(f'{i}. {display}')
print(f'{count:}, outgoing links\n')

# =====
# STEP 9: TOP 10 LOWEST OUT-DEGREE (but >0)
# =====

print("="*70)
print(" TOP 10 LEAST CENTRAL PAGES (Fewest Outgoing Links - but at least 1)")
print("="*70)

least_out = sorted(pages_with_out, key=lambda x: x[1])[:10]

for i, (url, count) in enumerate(least_out, 1):
    if 'ontariotechu.ca' in url:
        display = url.split('ontariotechu.ca')[1] if url.split('ontariotechu.ca')[1] else '/'
    elif 'uoit.ca' in url:
        display = url.split('uoit.ca')[1] if url.split('uoit.ca')[1] else '/'
    else:
        display = url[:80]
    print(f'{i}. {display}')
    print(f'{count} outgoing links\n')

# =====
# STEP 10: SAVE FULL RESULTS

```

```

# =====

print("="*70)
print(" Saving results...")

# Create results dataframe
results_data = []
for url in id_to_url.values():
    results_data.append({
        'URL': url,
        'In_Degree': in_degree_url.get(url, 0),
        'Out_Degree': out_degree_url.get(url, 0)
    })

results_df = pd.DataFrame(results_data)
results_df.to_csv(r"D:\complex_network\ontariotech_crawl\centrality_results.csv", index=False)

# Also save just the top ones
top_results = pd.DataFrame(top_in, columns=['URL', 'In_Degree'])
top_results.to_csv(r"D:\complex_network\ontariotech_crawl\top10_indegree.csv", index=False)

print(f" Full results saved to: centrality_results.csv")
print(f" Top 10 in-degree saved to: top10_indegree.csv")

# =====

# STEP 11: SUMMARY STATISTICS
# =====

print("\n" + "="*70)
print(" SUMMARY STATISTICS")
print("="*70)

```

```

print(f"Total nodes: {len(nodes_df):,}")
print(f"Total edges: {len(edges_df):,}")
print(f"Average in-degree: {total_in / len(nodes_df):.2f}")
print(f"Average out-degree: {total_out / len(nodes_df):.2f}")
print(f"Max in-degree: {max(in_degree_url.values()):,}")
print(f"Max out-degree: {max(out_degree_url.values()):,}")
print(f"Pages with zero incoming: {sum(1 for c in in_degree_url.values() if c == 0):,}")
print(f"Pages with zero outgoing: {sum(1 for c in out_degree_url.values() if c == 0):,}")

print(f"\n Processing time: {time.time()-start:.2f} seconds")

# =====
# WAIT 2 MINUTES BEFORE CLOSING
# =====

print("\n" + "="*70)
print(" SCRIPT COMPLETE!")
print("="*70)

print(" Results saved in: D:\\complex_network\\ontariotech_crawl\\")
print(" - centrality_results.csv (all nodes)")
print(" - top10_indegree.csv (top 10 only)")
print("\n Window will close automatically in 2 minutes...")
print(" Or press 'Ctrl + C' to close now.")

try:
    time.sleep(120) # Wait 120 seconds (2 minutes)
except KeyboardInterrupt:
    print("\n\n Window closed by user.")

```